Weekly Progress Report

Name	: NAGESHWARAN C
Domain	: CORE JAVA
Date of Submission	: 19-06-2026
Week Ending	: 04

I. Overview

Week 4 represented a major milestone in the Banking Information System project, with the project now standing at 80% completion. The primary focus this week was the successful integration of JDBC into the application, enabling persistent storage of customer and transaction data in a relational database. Alongside project work, advanced Java concepts including Stream API, Lambda Expressions, and design patterns were explored, significantly enhancing the codebase quality and readability.

II. Achievements

1. Project Development — Banking Information System (80% Completed)

- Successfully integrated JDBC into the Banking Information System — customer profiles and transaction records are now persisted in a MySQL relational database, replacing the previous in-memory storage.
- Implemented the Singleton design pattern for database connection management, ensuring a single shared connection instance throughout the application lifecycle.
- Applied the Factory design pattern for account creation, allowing dynamic instantiation of SavingsAccount and CurrentAccount objects based on user input.
- Completed the Account Closure feature, enabling users to deactivate accounts with appropriate validation and data archival.
- Implemented Fund Transfer functionality between accounts, including balance checks, atomic transaction handling, and rollback support on failure.
- Wrote JUnit test cases for all major modules — Account Management, Transaction Processing, and Customer Data Handling — achieving approximately 75% code coverage.
- Conducted code reviews with team members to resolve merge conflicts and maintain consistent code style across all modules.

2. Java Concepts Learned

- Stream API and Lambda Expressions: Deepened understanding of Java Streams for filtering, mapping, and reducing transaction collections. Applied lambda expressions to replace verbose anonymous class implementations, making the code more concise and readable.
- Design Patterns — Singleton and Factory: Studied and applied the Singleton pattern for the database connection manager and the Factory pattern for account object creation, improving project architecture and scalability.
- SQL Fundamentals: Studied core SQL — SELECT, INSERT, UPDATE, DELETE, and JOIN queries — to support JDBC-based database operations within the Banking System.
- JUnit Testing: Learned to write unit tests using JUnit 5, including use of @Test, @BeforeEach, @AfterEach annotations and assertion methods like assertEquals, assertThrows, and assertTrue.
- Exception Handling in JDBC: Practised structured exception handling for JDBC operations using try-with-resources to ensure connections and statements are always properly closed.

- PreparedStatement vs Statement: Understood the security and performance advantages of PreparedStatement over Statement, and applied it throughout the JDBC integration to prevent SQL injection.

3. Collaboration and Communication

- Participated in a team review session to demo the JDBC integration and gather feedback on database schema design.
- Collaborated with a team member to split JUnit test responsibilities across modules for faster coverage.
- Received mentor feedback on the Singleton implementation, leading to improvements in thread-safety handling for the database connection.
- Shared notes on Stream API and Lambda Expressions with the team, contributing to collective knowledge and readiness for future feature development.

III. Challenges

4. Project Complexity

- Managing database transactions for fund transfers required understanding of JDBC transaction control (commit/rollback), which needed careful implementation and testing to avoid data inconsistencies.
- Mapping existing in-memory data structures to relational database tables required thoughtful schema design to preserve application logic without breaking existing modules.
- Integrating JUnit tests into a project that was not initially designed with testability in mind required refactoring some tightly coupled classes to allow dependency injection.

5. Applying New Java Concepts

- Writing effective Stream pipelines for complex filtering and aggregation scenarios required multiple iterations to achieve clean and efficient solutions.
- Determining the appropriate design pattern for specific use cases required additional reading and discussion — particularly distinguishing when to use Factory versus Abstract Factory.

IV. Learning Resources

6. Java Learning Platforms

- Referred to the official Oracle Java documentation for Stream API, Lambda Expressions, and JUnit 5 specifications.
- Followed structured video tutorials on JDBC advanced topics including PreparedStatement, transactions, and connection pooling.
- Studied SQL fundamentals through online platforms and practised query writing to support JDBC integration.

7. Practical Application

- Applied Stream API and Lambda Expressions to real transaction data within the Banking System for filtering and generating summaries.
- Wrote isolated JDBC test programs to experiment with PreparedStatement and transaction handling before integrating into the main project.
- Implemented and tested JUnit test cases against all core modules to validate correctness and robustness.

V. Next Week’s Goals

8. Project Development

- Complete the remaining 20% of the Banking Information System — focusing on final feature refinements, end-to-end testing, and UI polish.
- Increase JUnit test coverage to above 90% by adding edge-case tests and integration tests for the JDBC layer.
- Prepare project documentation including a user guide, module overview, and database schema diagram for final submission.
- Conduct a full end-to-end system test to validate all features — account management, transactions, fund transfers, and account closure — work seamlessly together.

9. Java Learning Goals

- Explore Connection Pooling concepts (e.g., HikariCP) to understand how real-world applications manage database connections efficiently.
- Study advanced design patterns such as Observer and Strategy for potential application in future modules.
- Review and consolidate all Java concepts covered across the four weeks in preparation for final project evaluation.

VI. Additional Comments

Week 4 has been the most productive week of the internship, marked by the successful JDBC integration and the application of real-world design patterns within the Banking Information System. With the project at 80% completion, the finish line is clearly in sight. The combination of advancing Java concepts and applying them directly to the project has greatly accelerated both personal growth and team output. Mentor guidance on thread-safe Singleton implementation and overall architecture has been particularly insightful. The team is motivated and well-aligned to complete the final stretch in Week 5 with quality and thoroughness.